

APOSTILA DE COMPILADORES

Aula 11

Análise sintática ascendente

Parsing bottom-up, modelo shift-reduce, handles, prefixos viáveis e introdução aos analisadores LR.

Ciência da Computação — 2026/2
Professor: Alex Torquato Souza Carneiro
Projeto integrador: linguagem MINIC

Apresentação

Esta apostila acompanha a Aula 11 do curso de Compiladores. O foco é compreender como um analisador sintático ascendente reconstrói, a partir dos tokens, uma derivação mais à direita em ordem reversa. O estudo prepara a implementação do parser do projeto MINIC e introduz os fundamentos dos analisadores LR.

Objetivos de aprendizagem

- explicar a diferença entre análise descendente e ascendente;
- identificar handles e compreender a poda do handle;
- simular as ações shift, reduce, accept e error;
- relacionar pilha, prefixos viáveis e estados LR(0);
- distinguir LR(0), SLR(1), LR(1) e LALR(1);
- reconhecer e diagnosticar conflitos shift/reduce e reduce/reduce.

Relação com as aulas anteriores. A Aula 10 apresentou a análise LL(1) e o parser preditivo. Nesta aula, a direção da construção muda: em vez de expandir o símbolo inicial, o parser começa pelos tokens de entrada e reduz fragmentos até alcançar o símbolo inicial.

1. Visão geral da análise ascendente

A análise sintática ascendente constrói a árvore de derivação das folhas para a raiz. Para a entrada $id * id$, os tokens são agrupados em fatores, depois em termos e , finalmente, em uma expressão.

cap04_AH0g.qxd 26.10.07 14:34 Page 149



! Exercício 4.4.11: Modifique o seu algoritmo do Exercício 4.4.9 de modo que ele encontre, para qualquer cadeia, o menor número de erros de inserção, exclusão e mutação (cada erro com um único caractere) necessários para transformar a cadeia em uma cadeia na linguagem da gramática subjacente.

$stmt$	$\rightarrow$	if e then $stmt$ $stmtTail$
	$ $	while e do $stmt$
	$ $	begin $list$ end
	$ $	s
$stmtTail$	$\rightarrow$	else $stmt$
	$ $	ϵ
$list$	$\rightarrow$	$stmt$ $listTail$
$stmtTail$	$\rightarrow$	; $list$
	$\rightarrow$	ϵ

FIGURA 4.24 Uma gramática para certos tipos de comandos.

! Exercício 4.4.12: A Figura 4.24 mostra uma gramática para alguns comandos. Considere e e s como sendo terminais representando expressões condicionais e “outros comandos”, respectivamente. Se resolvermos o conflito considerando a expansão do “else” opcional (não-terminal $stmtTail$) preferindo consumir um **else** da entrada sempre que virmos um, podemos construir um analisador preditivo para essa gramática. Usando a idéia dos símbolos de sincronização descrita na Seção 4.4.5:

- Construa uma tabela do reconhecedor preditivo com correção de erro para a gramática.
- Mostre o comportamento do seu analisador nas seguintes entradas:
 - if** e **then** s ; **if** e **then** s **end**
 - while** e **do** **begin** s ; **if** e **then** s ; **end**

4.5 ANÁLISE ASCENDENTE

A análise sintática ascendente corresponde à construção de uma árvore de derivação para uma cadeia de entrada a partir das folhas (a parte de baixo) em direção à raiz (o topo) da árvore. Embora a árvore de derivação seja utilizada para descrever os métodos de análise, um *front-end* pode executar uma tradução diretamente, portanto, na prática, ela nunca é efetivamente construída. A sequência de instantâneos de árvore na Figura 4.25 ilustra o reconhecimento ascendente da sequência de tokens $id * id$, com respeito à gramática de expressão (4.1).

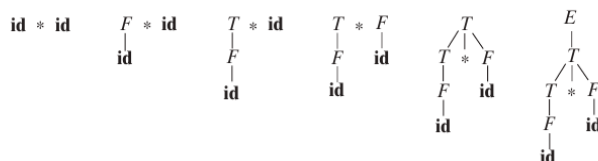


FIGURA 4.25 Uma análise ascendente para $id * id$.

Esta seção apresenta um método geral de análise ascendente conhecido como análise shift-reduce. A maior classe de gramáticas para a qual os analisadores sintáticos shift-reduce podem ser construídos, as gramáticas LR, será discutida nas seções 4.6 e 4.7. Embora seja muito trabalhoso construir um analisador sintático LR à mão, ferramentas chamadas geradores automáticos de analisadores sintáticos facilitam a construção de analisadores LR eficientes a partir de gramáticas adequadas. Nesta seção introduzimos alguns conceitos importantes para a definição de gramáticas apropriadas para fazer uso efetivo de um gerador de analisadores LR. A Seção 4.7 descreve os algoritmos usados para implementar geradores de analisadores sintáticos.

4.5.1 REDUÇÕES

Podemos pensar na análise ascendente como o processo de “reduzir” uma cadeia w para o símbolo inicial da gramática. Em cada passo da *redução*, uma subcadeia específica, casando com o lado direito de uma produção, é substituída pelo não-terminal na cabeça dessa produção.

As principais decisões relacionadas com a análise ascendente em cada passo do reconhecimento são: determinar quando reduzir e determinar a produção a ser usada para que a análise prossiga.

Figura 1 — Trecho do livro com a análise ascendente de $id * id$ e a introdução das reduções. Fonte: Aho, Sethi e Ullman, Cap. 4.

2. Reduções e derivação reversa

Uma redução substitui uma subcadeia que coincide com o lado direito de uma produção pelo não-terminal do lado esquerdo. Assim, a análise ascendente reconstrói uma derivação mais à direita ao contrário.

Gramática de expressões usada no exemplo

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid id \end{array}$$

Exemplo: a sequência de reduções para $id * id$ é:

$$id * id \Rightarrow F * id \Rightarrow T * id \Rightarrow T * F \Rightarrow T \Rightarrow E$$

A derivação correspondente, lida no sentido inverso, é $E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id$.

A redução correta não é apenas qualquer substituição possível. É preciso localizar o fragmento que corresponde ao próximo passo da derivação reversa: o handle.

3. Handle e poda do handle

Um handle de uma forma sentencial à direita é uma subcadeia que coincide com o corpo de uma produção e cuja redução produz a forma sentencial anterior na derivação mais à direita. Em uma gramática não ambígua, cada forma sentencial à direita possui um handle bem definido.

Forma sentencial	Handle	Redução
$id_1 * id_2$	id_1	$F \rightarrow id$
$F * id_2$	F	$T \rightarrow F$
$T * id_2$	id_2	$F \rightarrow id$
$T * F$	$T * F$	$E \rightarrow T * F$

Tabela 1 — Handles durante o reconhecimento de $id_1 * id_2$, adaptada do livro.

EXEMPLO 4.37: Os instantâneos da Figura 4.25 ilustram uma sequência de reduções usando a gramática de expressão (4.1). As reduções serão discutidas a partir da seguinte sequência de cadeias

$$\text{id} * \text{id}, F * \text{id}, T * \text{id}, T * F, T, E$$

As cadeias nessa sequência são formadas a partir das raízes de todas as subárvores nos instantâneos. A sequência começa com a cadeia de entrada $\text{id} * \text{id}$. A primeira redução produz $F * \text{id}$, reduzindo o id mais à esquerda para F , usando a produção $F \rightarrow \text{id}$. A segunda redução produz $T * \text{id}$, reduzindo F para T .

Neste ponto temos duas opções de redução: podemos reduzir a cadeia T para E , segundo a produção $E \rightarrow T$, e a cadeia consistindo no segundo id , que representa o lado direito de $F \rightarrow \text{id}$. Escolhemos a segunda opção e reduzimos id para F , produzindo a cadeia $T * F$. Essa cadeia é então reduzida para T . A análise termina com a redução de T para o símbolo inicial E .

Por definição, uma redução é o inverso de um passo em uma derivação (lembre-se de que, em uma derivação, um não-terminal em uma forma sentencial é substituído pelo lado direito de uma de suas produções). O objetivo do método de análise ascendente é, portanto, construir uma derivação ao reverso. A derivação a seguir corresponde à análise da Figura 4.25:

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \text{id} \Rightarrow F * \text{id} \Rightarrow \text{id} * \text{id}$$

Essa derivação é, na verdade, uma derivação mais à direita.

4.5.2 PODA DO HANDLE

A análise sintática ascendente ao ler a entrada, da esquerda para a direita, constrói uma derivação mais à direita ao inverso. Informalmente, um “handle” de uma cadeia de símbolos é uma subcadeia que casa com o corpo de uma produção, e cuja redução para o não-terminal do lado esquerdo representa um passo da derivação à direita ao inverso.

Por exemplo, incluindo subscritos aos tokens id por clareza, os handles durante a análise de $\text{id}_1 * \text{id}_2$ de acordo com a gramática da expressão (4.1) são ilustrados na Figura 4.26. Embora T seja o lado direito da produção $E \rightarrow T$, o símbolo T não é um handle na forma sentencial $T * \text{id}_2$. Se T fosse realmente substituído por E , obteríamos a cadeia $E * \text{id}_2$, a qual não pode ser derivada a partir do símbolo inicial E . Assim, a subcadeia mais à esquerda que casa com o corpo de alguma produção não precisa necessariamente ser um handle.

FORMA SENTENCIAL À DIREITA	HANDLE	PRODUÇÃO DE REDUÇÃO
$\text{id}_1 * \text{id}_2$	id_1	$F \rightarrow \text{id}$
$F * \text{id}_2$	F	$T \rightarrow F$
$T * \text{id}_2$	id_2	$F \rightarrow \text{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

FIGURA 4.26 Handles durante a análise de $\text{id}_1 * \text{id}_2$.

Formalmente, se $S \xRightarrow{r} \alpha A w \Rightarrow_r \alpha \beta w$, como na Figura 4.27, então a produção $A \rightarrow \beta$ na posição seguindo α é um handle de $\alpha \beta w$. Alternativamente, um handle de uma forma sentencial à direita γ é uma produção $A \rightarrow \beta$ e uma posição em γ onde a cadeia β pode ser localizada, de modo que a substituição de β nessa posição por A produz a forma sentencial à direita anterior em uma derivação mais à direita de γ .

Observe que a cadeia w à direita do handle deve conter apenas símbolos terminais. Por conveniência, nós nos referimos ao lado direito β em vez de $A \rightarrow \beta$ como um handle. Observe que dizemos “um handle”, em vez “o handle”, pois a gramática pode ser ambígua, com mais de uma derivação mais à direita para $\alpha \beta w$. Se uma gramática não for ambígua, então cada forma sentencial à direita da gramática tem exatamente um handle.

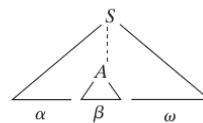


FIGURA 4.27 Um handle $A \rightarrow \beta$ na árvore de derivação para $\alpha \beta w$.

4. O analisador shift-reduce

O analisador shift-reduce mantém uma pilha de símbolos e um buffer com o restante da entrada. O handle, quando identificado, aparece no topo da pilha. O ciclo de análise alterna entre transferir tokens para a pilha e reduzir o handle.

- shift: empilha o próximo token e avança a entrada;
- reduce: remove o lado direito de uma produção e empilha o não-terminal correspondente;
- accept: indica que toda a entrada foi reconhecida;
- error: informa que não existe ação válida para a configuração atual.

Simulação resumida para $id * id \$$

Pilha	Entrada	Ação
\$	id * id \$	shift id
\$ id	* id \$	reduce $F \rightarrow id$
\$ F	* id \$	reduce $T \rightarrow F$
\$ T	* id \$	shift *
\$ T * id	\$	reduce $F \rightarrow id$; reduce $T \rightarrow T * F$
\$ E	\$	accept

Tabela 2 — Simulação didática; uma implementação real mantém também estados e constrói nós da AST.

5. Prefixos viáveis e estados LR(0)

Para decidir entre shift e reduce, o analisador precisa acompanhar o contexto sintático já reconhecido. Um prefixo viável é um prefixo de uma forma sentencial à direita que não ultrapassa o handle. Esses prefixos podem ser reconhecidos por um autômato finito determinístico.

Um item LR(0) representa uma produção com um ponto indicando quanto do lado direito já foi reconhecido:

$$[A \rightarrow \alpha \cdot \beta]$$

Se o ponto estiver antes de um símbolo, o parser pode avançar (GOTO) ao reconhecer esse símbolo. Se o ponto estiver no final, o item indica que uma redução pode estar disponível.

Operações fundamentais

- CLOSURE(I): acrescenta itens necessários quando o ponto antecede um não-terminal.
- GOTO(I, X): desloca o ponto sobre X e calcula o fechamento do conjunto resultante.
- Autômato LR(0): estados são conjuntos de itens; transições são obtidas por GOTO.

4.5.4 CONFLITOS DURANTE A ANÁLISE SINTÁTICA SHIFT-REDUCE

Existem gramáticas livres de contexto para as quais a análise shift-reduce não pode ser aplicada. Todo analisador sintático shift-reduce para tais gramáticas pode alcançar uma configuração na qual, conhecendo todo o conteúdo da pilha e o próximo símbolo da entrada, o analisador não é capaz de decidir se empilha e avança ou reduz, caracterizando um conflito conhecido como *conflito shift/reduce*, ou não consegue decidir qual das várias reduções disponíveis deverá ser aplicada, caracterizando um *conflito reduce/reduce*. A seguir são apresentados alguns exemplos de construções sintáticas que dão origem a tais gramáticas. Tecnicamente, essas gramáticas não pertencem à classe de gramáticas LR(k) definida na Seção 4.7; nós nos referimos a elas como gramáticas não-LR. O k em LR(k) refere-se ao número de símbolos à frente na entrada ainda não lidos, conhecidos como *lookaheads*. As gramáticas usadas na compilação usualmente pertencem à classe LR(1), e na prática apenas um símbolo à frente na entrada é suficiente.

EXEMPLO 4.38: Uma gramática ambígua nunca pode ser LR. Por exemplo, considere a gramática do *else* vazio (4.14) da Seção 4.3:

$$\begin{array}{l} stmt \rightarrow \text{if } expr \text{ then } stmt \\ \quad | \text{if } expr \text{ then } stmt \text{ else } stmt \\ \quad | \text{other} \end{array}$$

Se um analisador sintático shift-reduce em um momento tiver a configuração

PILHA	ENTRADA
... if <i>expr</i> then <i>stmt</i>	else ... \$

não há como saber se *if $expr$ then $stmt$* é um handle, não importa o que apareça abaixo dele na pilha. Essa configuração da pilha ilustra um conflito shift/reduce. Dependendo do que segue o *else* na entrada, pode ser correto reduzir *if $expr$ then $stmt$* para *stmt*, ou pode ser correto transferir o *else* para a pilha e então procurar outro *stmt* para completar a alternativa *if $expr$ then $stmt$ else $stmt$* .

Observe que a análise shift-reduce pode ser adaptada para reconhecer certas gramáticas ambíguas, como a gramática para o *if-then-else* mostrada. Se resolvermos o conflito shift/reduce do *else* em favor da transferência, o analisador se comportará conforme esperado, associando cada *else* com o *then* anterior mais próximo, ainda não casado. Discutimos os analisadores sintáticos para essas gramáticas ambíguas na Seção 4.8.

Outro cenário comum para conflitos ocorre quando sabemos que temos um handle, mas o conteúdo da pilha e o próximo símbolo da entrada são insuficientes para determinar qual produção deve ser usada em uma redução. O exemplo a seguir ilustra essa situação.

EXEMPLO 4.39: Suponha que um analisador léxico retorne *id* como o nome do token para todos os nomes, independentemente do seu tipo. Suponha também que nossa linguagem invoque procedimentos dando seus nomes, com parâmetros entre parênteses, e que os arranjos sejam referenciados com a mesma sintaxe. Como a tradução dos índices referenciados em arranjos e parâmetros nas chamadas de procedimento são diferentes, usamos produções diferentes para gerar as listas de parâmetros atuais e para os índices. Portanto, nossa gramática pode ter, entre outras, as produções apresentadas na Figura 4.30.

(1)	<i>stmt</i>	$\rightarrow$	<i>id</i> (<i>parameter-list</i>)
(2)	<i>stmt</i>	$\rightarrow$	<i>expr</i> := <i>expr</i>
(3)	<i>parameter_list</i>	$\rightarrow$	<i>parameter_list</i> , <i>parameter</i>
(4)	<i>parameter_list</i>	$\rightarrow$	<i>parameter</i>
(5)	<i>parameter</i>	$\rightarrow$	<i>id</i>
(6)	<i>expr</i>	$\rightarrow$	<i>id</i> (<i>expr_list</i>)
(7)	<i>expr</i>	$\rightarrow$	<i>id</i>
(8)	<i>expr_list</i>	$\rightarrow$	<i>expr_list</i> , <i>expr</i>
(9)	<i>expr_list</i>	$\rightarrow$	<i>expr</i>

FIGURA 4.30 Produções envolvendo chamadas de procedimento e referências a índices de arranjos.

Um comando começando com $p(i, j)$ é representado internamente pela sequência de tokens *id(id, id)* durante a análise sintática. Após transferir os três primeiros tokens para a pilha, um analisador shift-reduce estaria na configuração

PILHA	ENTRADA
... id (id	, id) ...

Figura 3 — Página do livro com a discussão dos conflitos e a introdução dos itens e do autômato LR(0). Fonte: Aho, Sethi e Ullman, Cap. 4.

6. SLR(1), LR(1) e LALR(1)

A família LR reúne métodos que usam uma pilha de estados e tabelas para escolher ações sem retrocesso. A diferença principal entre as variantes está na quantidade de contexto mantida e no tamanho das tabelas.

Método	Ideia central	Característica
LR(0)	Itens sem lookahead	Mais simples; pode gerar muitos conflitos
SLR(1)	FOLLOW para reduções	Usa autômato LR(0) com contexto global
LR(1)	Lookahead nos itens	Mais poderoso; tabelas maiores
LALR(1)	Combina estados compatíveis	Boa relação entre poder e tamanho

Tabela 3 — Comparação introdutória das famílias LR.

SLR(1). Quando um item completo $A \rightarrow \alpha$ é alcançado, a redução é inserida nas colunas correspondentes a FOLLOW(A). Se uma célula receber duas ações, há conflito.

Uso prático. Geradores de parser frequentemente produzem tabelas LALR(1) ou LR(1), além de relatarem conflitos para que a gramática ou as prioridades sejam revistas.

7. Conflitos e diagnóstico

Conflito shift/reduce: na mesma configuração, o parser não sabe se deve transferir o próximo token ou reduzir o conteúdo da pilha. O caso clássico é o if-then-else, quando o token else pode ser associado ao comando condicional mais próximo.

Conflito reduce/reduce: duas reduções diferentes parecem possíveis. Em geral, indica uma gramática ambígua, uma especificação incompleta ou uma escolha de contexto insuficiente.



Uma derivação mais à direita ao reverso pode ser obtida pela “poda do handle”. Ou seja, começamos com uma cadeia de terminais w a ser analisada. Se w for uma sentença da gramática dada, então considere $w = \gamma_n$, onde γ_n é a n -ésima forma sentencial mais à direita de alguma derivação mais à direita ainda desconhecida.

$$S = \gamma_0 \Rightarrow_{rm} \gamma_1 \Rightarrow_{rm} \gamma_2 \Rightarrow_{rm} \dots \Rightarrow_{rm} \gamma_{n-1} \Rightarrow_{rm} \gamma_n = w$$

Para reconstruir essa derivação em ordem reversa, localizamos o handle β_n em γ_n e substituímos β_n pelo lado esquerdo de alguma produção relevante $A_n \rightarrow \beta_n$ para obter a forma sentencial anterior mais à direita γ_{n-1} . Observe que ainda não sabemos como encontrar os handles, mas veremos métodos para reconhecê-los em breve.

Repetimos esse processo, ou seja, localizamos o handle β_{n-1} em γ_{n-1} e reduzimos esse handle para obter a forma sentencial mais à direita γ_{n-2} . Se continuarmos esse processo e conseguirmos alcançar o símbolo inicial da gramática, paramos e anunciamos sucesso no reconhecimento. O reverso da sequência de produções usadas nas reduções é uma derivação mais à direita para a cadeia da entrada.

4.5.3 ANALISADOR SINTÁTICO SHIFT-REDUCE

O analisador sintático shift-reduce é uma forma de análise ascendente em que uma pilha contém símbolos da gramática e um buffer de entrada contém o restante da cadeia a ser reconhecida sintaticamente. Conforme veremos, o handle sempre aparece no topo da pilha, imediatamente antes de ser identificado como handle.

Usamos o símbolo \$ para marcar o fundo da pilha e também o extremo direito da entrada. Adotaremos a seguinte convenção ao discutirmos a análise ascendente: mostramos o topo da pilha à direita, em vez de à esquerda, como fizemos para a análise descendente. Inicialmente, a pilha está vazia, e a cadeia w representa a entrada, da seguinte forma:

PILHA	ENTRADA
\$	w \$

Durante uma escansão da esquerda para a direita da cadeia de entrada, o analisador sintático transfere⁴ zero ou mais símbolos da entrada para a pilha, até que uma cadeia β de símbolos da gramática presente no topo da pilha possa ser reduzida para o lado esquerdo da produção apropriada. O analisador repete esse ciclo até detectar um erro na entrada ou até que a pilha contenha apenas o símbolo inicial da gramática e a entrada esteja vazia:

PILHA	ENTRADA
\$ S	\$

Ao entrar nessa configuração, o analisador pára e anuncia sucesso no reconhecimento sintático. A Figura 4.28 mostra as ações do analisador shift-reduce durante o reconhecimento da cadeia de entrada $\text{id}_1 * \text{id}_2$ de acordo com a gramática da expressão (4.1).

PILHA	ENTRADA	AÇÃO
\$	$\text{id}_1 * \text{id}_2$ \$	transfere
\$ id_1	$* \text{id}_2$ \$	reduz segundo $F \rightarrow \text{id}$
\$ F	$* \text{id}_2$ \$	reduz segundo $T \rightarrow F$
\$ T	$* \text{id}_2$ \$	transfere
\$ $T *$	id_2 \$	transfere
\$ $T * \text{id}_2$	\$	reduz segundo $F \rightarrow \text{id}$
\$ $T * F$	\$	reduz segundo $T \rightarrow T * F$
\$ T	\$	reduz segundo $E \rightarrow T$
\$ E	\$	aceita

FIGURA 4.28 Configurações de um analisador shift-reduce sob a entrada $\text{id}_1 * \text{id}_2$.

Embora as principais operações sejam shift e reduce, na realidade existem quatro ações possíveis que um analisador shift-reduce pode realizar: (1) shift, (2) reduce, (3) accept e (4) error.

1. *Shift*. Transfere o próximo símbolo da entrada para o topo da pilha.
2. *Reduce*. O extremo direito da cadeia a ser reduzida deve estar no topo da pilha. Localize o extremo esquerdo da cadeia no interior da pilha e decida por qual não-terminal esta cadeia será substituída.

4 N. do R.T.: Do inglês *shift*, que significa “empilha e avança”.

8. Aplicação ao parser da MINIC

No projeto MINIC, a análise ascendente pode ser usada para reconhecer declarações, funções, blocos, comandos e expressões. A integração com o scanner fornece uma sequência de tokens com lexema, linha e coluna.

- a pilha de estados registra o contexto sintático;
- as reduções podem criar diretamente nós da AST;
- cada nó deve preservar posição e, quando necessário, o lexema original;
- erros devem informar token inesperado, posição e contexto esperado;
- a tabela do parser deve ser testada com entradas válidas e inválidas.

Exemplo de construção de AST durante uma redução

Ao reduzir $F \rightarrow id$, o parser pode criar um nó Identificador contendo o lexema e sua posição. Ao reduzir $T \rightarrow T * F$, cria um nó BinOp com operador “*”, ligando os nós esquerdo e direito. A AST não precisa reproduzir todos os símbolos da gramática: ela deve representar a estrutura relevante para as fases semântica e de geração de código.

Checklist de implementação

- scanner retorna tokens consistentes;
- gramática está documentada e sem ambiguidades não resolvidas;
- estados e transições foram verificados;
- ações shift/reduce estão cobertas por testes;
- AST e mensagens de erro foram validadas.

9. Atividades práticas

Atividade 1 — Reduções. Para a gramática $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F$ e $F \rightarrow \text{id}$, indique uma sequência de reduções para $\text{id} + \text{id} * \text{id}$. Explique por que a multiplicação deve ser reduzida antes da adição.

Atividade 2 — Shift-reduce. Monte uma tabela com pilha, entrada e ação para reconhecer $\text{id} * \text{id}$. Inclua as ações accept e error.

Atividade 3 — Itens LR(0). Construa o fechamento do conjunto inicial da gramática estendida $E' \rightarrow E$ e indique as transições GOTO para os símbolos possíveis.

Atividade 4 — Conflitos. Analise a gramática do if-then-else. Identifique a configuração de conflito e proponha uma regra de desempate.

Atividade 5 — MINIC. Integre um parser ao scanner e registre a AST textual para uma função simples com declaração, atribuição e retorno. Teste pelo menos três entradas inválidas.

10. Síntese

A análise ascendente começa pelos tokens e busca reconstruir a derivação até o símbolo inicial. O modelo shift-reduce organiza esse processo em uma pilha e quatro ações possíveis. O conceito de handle explica qual fragmento deve ser reduzido; prefixos viáveis e itens LR(0) fornecem a base para decidir as ações. SLR(1), LR(1) e LALR(1) ampliam o contexto disponível e permitem reconhecer classes maiores de gramáticas.

Leitura orientada Aho, Sethi e Ullman, Compiladores: princípios, técnicas e ferramentas, Capítulo 4, Seções 4.5 e 4.6.